

CAHIER DES CHARGES

Portfolio Développeur Fullstack

Architecture Headless : Vue.js + Django REST API

Profil	Développeur Junior Fullstack
Stack Frontend	Vue.js 3 + Tailwind CSS
Stack Backend	Django + Django REST Framework
Base de données	PostgreSQL
Hébergement Front	Vercel
Hébergement Back	Heroku
Date	10/02/2026

TABLE DES MATIÈRES

- 1. Contexte et Objectifs du Projet
- 2. Architecture Technique
- 3. Spécifications Fonctionnelles
- 4. Design et Interface Utilisateur
- 5. Système d'Authentification
- 6. Gestion des Données et API
- 7. Déploiement et Hébergement
- 8. Feuille de Route
- 9. Points d'Attention Critiques
- 10. Évolutions Futures

1. CONTEXTE ET OBJECTIFS DU PROJET

1.1 Contexte

Ce projet consiste en la création d'un portfolio professionnel dynamique pour un développeur junior fullstack. Le site servira de vitrine professionnelle démontrant les compétences techniques en développement frontend et backend.

1.2 Objectifs Principaux

- **Vitrine professionnelle** : Démontrer les compétences techniques en développement web moderne
- **Architecture moderne** : Implémenter une architecture headless (découplée) Frontend/Backend
- **Gestion dynamique** : Permettre l'ajout, la modification et la suppression de projets via une interface d'administration
- **Conversion** : Attirer et convertir des prospects (recruteurs techniques et PME)
- **Apprentissage** : Se familiariser avec les pratiques professionnelles actuelles (API REST, déploiement hybride)

1.3 Public Cible

- **Recruteurs techniques** : CTO, Lead Developers, Responsables RH tech
- **Clients PME** : Petites et moyennes entreprises recherchant des solutions web
- **Collaborateurs potentiels** : Autres développeurs pour des projets collaboratifs

2. ARCHITECTURE TECHNIQUE

2.1 Vue d'Ensemble de l'Architecture

Le projet adopte une architecture headless (découplée) séparant strictement le frontend du backend. Cette approche reflète les pratiques modernes utilisées par les grandes entreprises tech (Netflix, Airbnb, Spotify).

Composant	Technologie	Hébergement	Rôle
Frontend	Vue.js 3 + Vite	Vercel	Interface utilisateur
Backend	Django + DRF	Heroku	API REST + Logique métier
Base de données	PostgreSQL	Heroku (Add-on)	Stockage des données
Stockage médias	Cloudinary	Cloud	Images des projets

2.2 Stack Backend (Django)

- **Framework** : Django 4.x
- **API** : Django REST Framework (DRF) - pour la sérialisation JSON
- **Base de données** : PostgreSQL (production) / SQLite (développement)
- **Authentification** : Django REST Framework Token Authentication ou JWT
- **CORS** : django-cors-headers (essentiel pour l'architecture headless)
- **Stockage médias** : django-cloudinary-storage (images persistantes)
- **Serveur WSGI** : Gunicorn (pour Heroku)

2.3 Stack Frontend (Vue.js)

- **Framework** : Vue.js 3 (Composition API)
- **Build Tool** : Vite (rapide et moderne)
- **Styling** : Tailwind CSS 3.x
- **Requêtes HTTP** : Axios (communication avec l'API Django)
- **Routage** : Vue Router (navigation SPA)
- **Gestion d'état** : Pinia (optionnel, si nécessaire)
- **Animations** : AOS (Animate On Scroll) pour les effets au défilement

2.4 Communication Frontend ↔ Backend

Le frontend Vue.js communique avec le backend Django exclusivement via des requêtes HTTP vers l'API REST. Django ne sert aucun HTML, uniquement du JSON.

Exemple de flux : 1. L'utilisateur visite la page Portfolio (Vue.js sur Vercel) 2. Le composant Vue envoie une requête GET via Axios : GET <https://mon-portfolio-api.herokuapp.com/api/projets/> 3. Django DRF renvoie les données au format JSON 4. Vue.js affiche les projets dans l'interface

3. SPÉCIFICATIONS FONCTIONNELLES

3.1 Arborescence du Site

Page	URL	Description	Données dynamiques
Accueil	/	Hero, projets récents, CTA	Oui (3 derniers projets)
Portfolio	/portfolio	Liste complète des projets	Oui (tous les projets)
Projet Détail	/portfolio/:id	Détail d'un projet	Oui (projet spécifique)
Services	/services	Offres pour PME	Non (contenu statique)
Contact	/contact	Formulaire de contact	Non

3.2 Page d'Accueil (Home)

- **Hero Section** : Phrase d'accroche professionnelle avec animation d'entrée
- **Stack Technologies** : Logos des technologies maîtrisées (badges animés)
- **Projets Récents** : Cartes des 3 derniers projets (récupérés via API)
- **Call-to-Action (CTA)** : Bouton 'Discuter de votre projet' menant au formulaire de contact
- **Statistiques** : Nombre de projets réalisés, technologies maîtrisées (compteurs animés)

3.3 Page Portfolio

- **Grille de projets** : Affichage en cartes (Grid responsive)
- **Filtres dynamiques** : Par catégorie (Sites Vitrites, Applications Web, Scripts Python)
- **Recherche** : Barre de recherche par nom de projet ou technologie
- **Tri** : Par date (plus récent/plus ancien), par popularité
- **Animation** : Effet 'Fade Up' au scroll (AOS library)

3.4 Page Détail Projet

- **Galerie d'images** : Carousel ou lightbox pour les captures d'écran
- **Description complète** : Contexte, problématique, solution apportée
- **Technologies utilisées** : Liste avec badges colorés
- **Liens** : Démo live (si applicable) + Dépôt GitHub
- **Défis techniques** : Section expliquant les difficultés rencontrées
- **Navigation** : Boutons 'Projet précédent' / 'Projet suivant'

3.5 Page Services (Spécial PME)

- **Développement Web** : Sites vitrines, applications Django
- **Automatisation** : Scripts Python, web scraping, traitement de données
- **Maintenance** : Audit de code, correction de bugs, mises à jour
- **Process** : Explication de la méthodologie de travail
- **Tarification** : Grille indicative ou lien vers devis

3.6 Page Contact

- **Formulaire** : Nom, Email, Type de projet (dropdown), Budget estimé, Message
- **Validation** : Validation côté client (Vue.js) et côté serveur (Django)
- **Envoi d'email** : Notification par email lors de la soumission (SendGrid ou Brevo)
- **Confirmation** : Message de succès après envoi
- **Réseaux sociaux** : Liens vers GitHub, LinkedIn, Twitter

4. DESIGN ET INTERFACE UTILISATEUR

4.1 Charte Graphique

Le design s'inspire d'une esthétique Modern Tech / Clean Cyberpunk avec des tons verts et noirs pour évoquer la technologie et le code.

Élément	Couleur	Code Hex	Usage
Fond principal	Obsidian Black	#0a0a0a	Arrière-plan général
Fond secondaire	Dark Jungle	#121e16	Cartes, sections
Couleur primaire	Emerald	#10b981	Titres, liens, marque
Couleur accent	Neon Lime	#d9f99d	CTA, boutons importants
Texte principal	Off White	#f9f9f9	Corps de texte
Texte secondaire	Gray	#9ca3af	Sous-titres, métadonnées

4.2 Typographie

- **Police principale** : Inter (Google Fonts) - moderne et lisible
- **Police technique** : Space Grotesk ou JetBrains Mono pour le code
- **Hiérarchie** : Titres en Bold, corps en Regular, code en Monospace

4.3 Modes d'Affichage

Dark Mode / Light Mode : Toggle dans le header permettant de basculer entre les deux modes. Tailwind CSS gère nativement le dark mode avec le préfixe dark:.

4.4 Animations et Interactions

- **Scroll Animations** : AOS (Animate On Scroll) pour les effets d'apparition
- **Hover Effects** : Transformation des cartes au survol (scale, shadow)
- **Transitions** : Transitions fluides entre les pages (Vue Router)
- **Loading States** : Skeletons ou spinners pendant le chargement des données API
- **Micro-interactions** : Feedback visuel sur les boutons (ripple effect)

4.5 Responsive Design

Le site doit être 100% responsive et optimisé pour :

- **Mobile (320px - 768px)** : Navigation burger, grille 1 colonne
- **Tablette (768px - 1024px)** : Grille 2 colonnes
- **Desktop (1024px+)** : Grille 3-4 colonnes, navigation horizontale

5. SYSTÈME D'AUTHENTIFICATION

5.1 Objectif

Seul l'administrateur (vous) pourra ajouter, modifier ou supprimer des projets via une interface dédiée. Le grand public ne pourra que consulter le contenu.

5.2 Solutions Techniques Recommandées

Option 1 : Django Admin natif (Recommandé pour MVP)

- **Avantages** : Déjà intégré, zéro développement frontend nécessaire
- **Accès** : `https://mon-api.herokuapp.com/admin/`
- **Sécurité** : Authentification Django native, très robuste
- **Personnalisation** : Interface personnalisable via les classes `ModelAdmin`

Option 2 : Interface admin Vue.js personnalisée

- **Développement** : Créer une route `/admin` dans `Vue.js`
- **Authentification** : JWT (JSON Web Tokens) avec DRF
- **Flux** : Login → Récupération du token → Requêtes authentifiées vers l'API
- **Complexité** : Plus de travail mais meilleure expérience utilisateur
- **Sécurité** : Protection des routes `Vue.js` + validation backend

5.3 Implémentation Recommandée (Django Admin)

- Créer un superutilisateur : `python manage.py createsuperuser`
- Enregistrer les modèles dans `admin.py`
- Personnaliser l'affichage (colonnes, filtres, recherche)
- Déployer sur Heroku et accéder via `/admin/`
- Utiliser des permissions Django pour gérer l'accès si plusieurs administrateurs

5.4 Sécurité

- **HTTPS** : Obligatoire en production (automatique sur Heroku et Vercel)
- **CSRF Protection** : Django CSRF tokens pour les formulaires
- **Rate Limiting** : Limiter les tentatives de connexion (`django-ratelimit`)
- **Mots de passe forts** : Utiliser des mots de passe complexes pour l'admin
- **Variables d'environnement** : Ne jamais exposer les secrets (`SECRET_KEY`, DB credentials)

6. GESTION DES DONNÉES ET API

6.1 Modèles de Données (Django)

Modèle Projet

Représente chaque projet du portfolio :

- titre (CharField, max 200 caractères)
- slug (SlugField, unique, pour les URLs)
- description_courte (TextField, 200 caractères)
- description_longue (TextField, description complète)
- image_principale (ImageField, stockée sur Cloudinary)
- galerie_images (relation Many-to-Many avec un modèle ImageProjet)
- technologies (ManyToManyField avec modèle Technologie)
- categorie (ForeignKey: Site Vitrine, App Web, Script)
- lien_github (URLField, optionnel)
- lien_demo (URLField, optionnel)
- date_creation (DateField, auto_now_add=True)
- date_mise_a_jour (DateField, auto_now=True)
- ordre_affichage (IntegerField, pour tri manuel)
- est_publie (BooleanField, permet de masquer un projet)

Modèle Contact

- nom (CharField)
- email (EmailField)
- type_projet (CharField avec choices: Site vitrine, App web, etc.)
- budget (CharField avec ranges)
- message (TextField)
- date_envoi (DateTimeField, auto_now_add=True)
- traite (BooleanField, pour marquer les messages traités)

6.2 Endpoints API (Django REST Framework)

L'API REST expose les données au format JSON pour consommation par Vue.js :

Méthode	Endpoint	Description	Auth
GET	/api/projets/	Liste tous les projets publiés	Non
GET	/api/projets/<id>/	Détail d'un projet	Non
POST	/api/projets/	Créer un projet	Oui (Admin)
PUT/PATCH	/api/projets/<id>/	Modifier un projet	Oui (Admin)
DELETE	/api/projets/<id>/	Supprimer un projet	Oui (Admin)
POST	/api/contact/	Envoyer un message	Non
GET	/api/technologies/	Liste des technos	Non

GET	/api/categories/	Liste des catégories	Non
-----	------------------	----------------------	-----

6.3 Sérialseurs (Serializers)

Les sérialiseurs DRF transforment les modèles Django en JSON et vice-versa. Chaque modèle principal aura son sérialiseur correspondant.

6.4 Pagination et Filtrage

- **Pagination** : PageNumberPagination (ex: 12 projets par page)
- **Filtres** : django-filter pour filtrer par catégorie, technologie
- **Recherche** : SearchFilter pour recherche textuelle
- **Tri** : OrderingFilter pour trier par date, popularité

7. DÉPLOIEMENT ET HÉBERGEMENT

7.1 Hébergement Backend (Heroku)

Fichiers nécessaires :

- requirements.txt (dépendances Python)
- Procfile (commande de démarrage : web: gunicorn config.wsgi)
- runtime.txt (version Python : python-3.11.x)
- .env (variables d'environnement, ne pas commit)

Add-ons Heroku :

- Heroku Postgres (base de données)
- Cloudinary (optionnel si géré autrement)

Variables d'environnement :

- SECRET_KEY
- DATABASE_URL (auto-configuré par Postgres add-on)
- ALLOWED_HOSTS
- DEBUG=False
- CORS_ALLOWED_ORIGINS (URL de Vercel)

Processus de déploiement :

- Créer une app Heroku : `heroku create mon-portfolio-api`
- Ajouter le add-on PostgreSQL : `heroku addons:create heroku-postgresql:mini`
- Configurer les variables d'environnement : `heroku config:set SECRET_KEY=...`
- Déployer via Git : `git push heroku main`
- Migrer la base de données : `heroku run python manage.py migrate`
- Créer le superutilisateur : `heroku run python manage.py createsuperuser`
- Collecter les fichiers statiques : `heroku run python manage.py collectstatic`

7.2 Hébergement Frontend (Vercel)

- Connecter le dépôt GitHub à Vercel
- Vercel détecte automatiquement Vite
- Configurer la variable d'environnement VITE_API_URL (URL Heroku)
- Déploiement automatique à chaque push sur main

Build settings :

- Framework Preset : Vite
- Build Command : `npm run build`
- Output Directory : `dist`
- Install Command : `npm install`

7.3 Stockage des Images (Cloudinary)

Problématique : Heroku utilise un système de fichiers éphémère. Les images uploadées sont supprimées lors des redémarrages (toutes les 24h).

Solution :

- Créer un compte gratuit sur Cloudinary
- Installer : `pip install django-cloudinary-storage`
- Configurer dans `settings.py` avec les credentials Cloudinary
- Les images seront automatiquement uploadées sur le CDN Cloudinary
- URLs des images générées automatiquement par Django

7.4 Nom de Domaine Personnalisé (Optionnel)

- Acheter un domaine (ex: `www.mon-portfolio.com` sur Namecheap, OVH)
- Configurer les DNS pour pointer vers Vercel (frontend)
- Configurer un sous-domaine `api.mon-portfolio.com` pour Heroku (backend)
- Mettre à jour `CORS_ALLOWED_ORIGINS` avec le nouveau domaine

8. FEUILLE DE ROUTE

8.1 Phase 1 : Backend (Django + API)

1. Initialiser le projet Django : `django-admin startproject config` .
2. Créer l'app principale : `python manage.py startapp portfolio`
3. Installer Django REST Framework et `django-cors-headers`
4. Créer les modèles (Projet, Technologie, Catégorie, Contact)
5. Créer les migrations et migrer : `python manage.py makemigrations` & `python manage.py migrate`
6. Créer les sérialiseurs dans `serializers.py`
7. Créer les ViewSets dans `views.py`
8. Configurer les routes dans `urls.py`
9. Configurer CORS pour autoriser les requêtes depuis localhost (dev)
10. Tester l'API localement avec Postman ou Insomnia
11. Créer un superutilisateur et ajouter du contenu de test
12. Préparer le déploiement Heroku (Procfile, requirements.txt, runtime.txt)
13. Déployer sur Heroku et tester les endpoints en production

8.2 Phase 2 : Frontend (Vue.js)

1. Créer le projet Vue : `npm create vite@latest portfolio-front -- --template vue`
2. Installer Tailwind CSS : `npm install -D tailwindcss postcss autoprefixer`
3. Configurer Tailwind (`tailwind.config.js`, styles globaux)
4. Installer Vue Router : `npm install vue-router@4`
5. Installer Axios : `npm install axios`
6. Créer la structure de dossiers (`components/`, `views/`, `services/`, `router/`)
7. Développer la page d'accueil (statique d'abord)
8. Créer un service API Axios (axios instance avec baseURL)
9. Développer la page Portfolio avec appel API
10. Développer la page détail Projet
11. Développer la page Contact avec formulaire
12. Implémenter le Dark/Light mode toggle
13. Ajouter les animations AOS
14. Optimiser le responsive design
15. Tester localement avec l'API Heroku
16. Déployer sur Vercel et mettre à jour `CORS_ALLOWED_ORIGINS`

8.3 Phase 3 : Améliorations et Optimisations

1. Optimiser les images (compression, lazy loading)
2. Implémenter un système de cache (Redis sur Heroku)
3. Ajouter Google Analytics ou Plausible pour le tracking
4. Optimiser le SEO (meta tags, Open Graph, sitemap.xml)

5. Ajouter des tests unitaires (backend: pytest, frontend: Vitest)
6. Configurer CI/CD (GitHub Actions pour tests automatiques)
7. Améliorer l'accessibilité (ARIA labels, navigation clavier)
8. Ajouter une page 404 personnalisée

8.4 Timeline Estimée

Phase	Durée estimée	Livrables
Phase 1 (Backend)	2-3 semaines	API fonctionnelle sur Heroku
Phase 2 (Frontend)	3-4 semaines	Site complet déployé sur Vercel
Phase 3 (Optimisations)	1-2 semaines	Site optimisé et production-ready
Total	6-9 semaines	Portfolio professionnel complet

9. POINTS D'ATTENTION CRITIQUES

9.1 Configuration CORS (Essentiel)

Problème : Par défaut, les navigateurs bloquent les requêtes entre différents domaines (Vercel → Heroku) pour des raisons de sécurité.

Solution :

- Installer : `pip install django-cors-headers`
- Ajouter 'corsheaders' dans `INSTALLED_APPS`
- Ajouter le middleware : `'corsheaders.middleware.CorsMiddleware'`
- Configurer `CORS_ALLOWED_ORIGINS` avec l'URL Vercel exacte
- En développement, autoriser `localhost:5173` (Vite dev server)

9.2 Gestion des Images (Système de Fichiers Éphémère)

Piège : Heroku supprime les fichiers uploadés à chaque redémarrage (toutes les 24h).

Solution obligatoire : Utiliser Cloudinary ou AWS S3 pour le stockage persistant.

9.3 Variables d'Environnement

- Ne jamais commiter les fichiers `.env` sur Git
- Utiliser `python-decouple` ou `django-environ` pour gérer les variables
- Configurer les variables sur Heroku avec `heroku config:set`
- Sur Vercel, configurer via l'interface web (Settings → Environment Variables)

9.4 Performance et Optimisation

- **Images** : Utiliser WebP, lazy loading, `srcset` pour responsive images
- **API** : Implémenter la pagination pour éviter de charger tous les projets
- **Cache** : Utiliser le cache Django pour les requêtes fréquentes
- **CDN** : Cloudinary sert les images via CDN (rapide partout dans le monde)
- **Bundle size** : Analyser le bundle Vue.js (`rollup-plugin-visualizer`)

9.5 SEO et Référencement

- **Meta tags** : Utiliser `vue-meta` ou `@vueuse/head` pour gérer les meta tags
- **SSR/SSG** : Considérer `Nuxt.js` (SSR) si le SEO est critique (complexité++)
- **Sitemap** : Générer un `sitemap.xml` (`django-sitemap` pour l'API)
- **Open Graph** : Ajouter les tags OG pour le partage social
- **Lighthouse** : Viser un score > 90 (Performance, SEO, Accessibility)

9.6 Sécurité

- **HTTPS** : Obligatoire (automatique sur Heroku et Vercel)
- **CSRF** : Django gère automatiquement pour les forms
- **SQL Injection** : Django ORM protège automatiquement

- **XSS** : Vue.js échappe automatiquement le HTML, utiliser v-html avec précaution
- **Rate Limiting** : Protéger l'endpoint contact contre le spam
- **Secrets** : Utiliser des mots de passe forts, activer 2FA sur Heroku/Vercel

10. ÉVOLUTIONS FUTURES

10.1 Fonctionnalités Court Terme

- **Recherche avancée** : Recherche full-text avec Elasticsearch ou Algolia
- **Statistiques** : Dashboard admin avec stats (vues, projets populaires)
- **Multilingue** : Support FR/EN avec vue-i18n

10.2 Fonctionnalités Long Terme

- **Espace client** : Portail client pour suivre les projets en cours
- **Système de devis** : Générateur de devis automatique
- **API publique** : Exposer une API publique pour d'autres devs
- **PWA** : Transformer le site en Progressive Web App
- **Tests E2E** : Tests end-to-end avec Playwright ou Cypress

10.3 Technologies à Surveiller

- **Nuxt.js** : Pour améliorer le SEO avec SSR
- **GraphQL** : Alternative à REST API (plus flexible)
- **WebSockets** : Pour des fonctionnalités temps réel
- **Docker** : Conteneurisation pour faciliter le déploiement
- **Kubernetes** : Orchestration si le trafic augmente significativement

CONCLUSION

Ce cahier des charges définit un projet ambitieux mais réaliste pour un développeur junior. L'architecture headless Vue.js + Django vous permettra d'acquérir des compétences hautement valorisées sur le marché du travail.

- L'architecture est séparée : le frontend (Vue.js) et le backend (Django) communiquent uniquement via API REST
- La configuration CORS est critique pour que le tout fonctionne
- Cloudinary est indispensable pour le stockage des images sur Heroku
- Commencez par le backend (API fonctionnelle) avant d'attaquer le frontend
- Utilisez Django Admin pour le MVP, une interface custom peut venir plus tard
- Déployez tôt et souvent pour identifier les problèmes rapidement

Prochaine étape : Initialiser le projet Django et créer les premiers modèles de données. Bon développement !